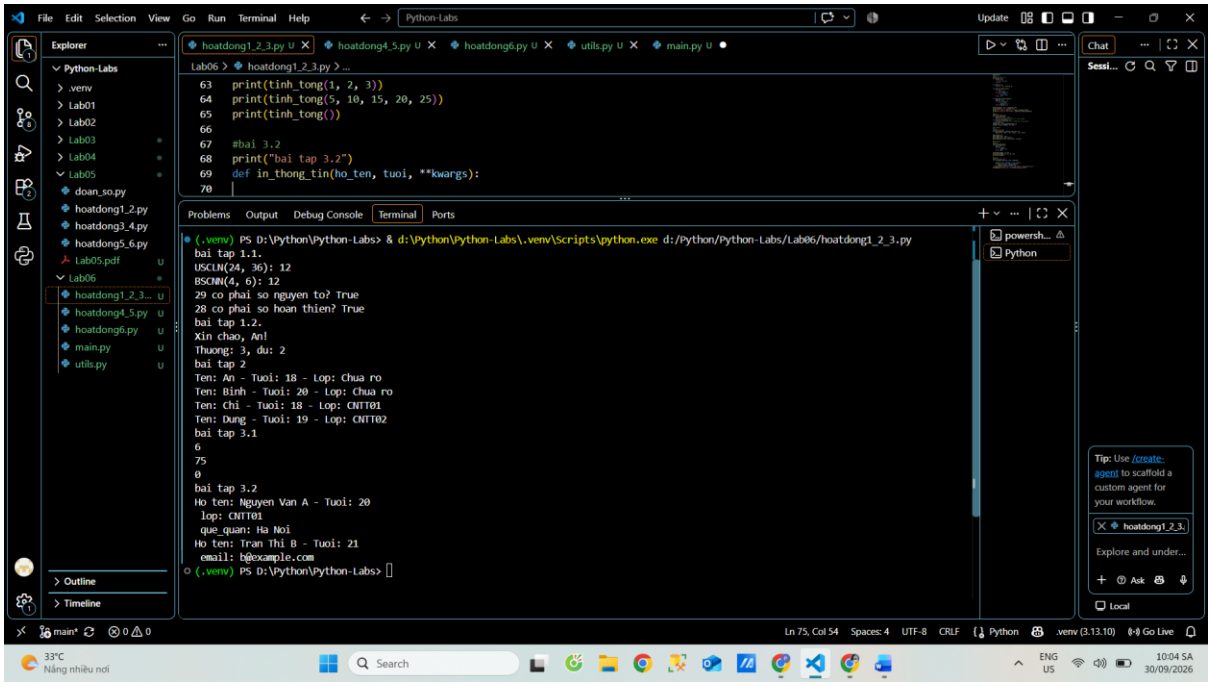
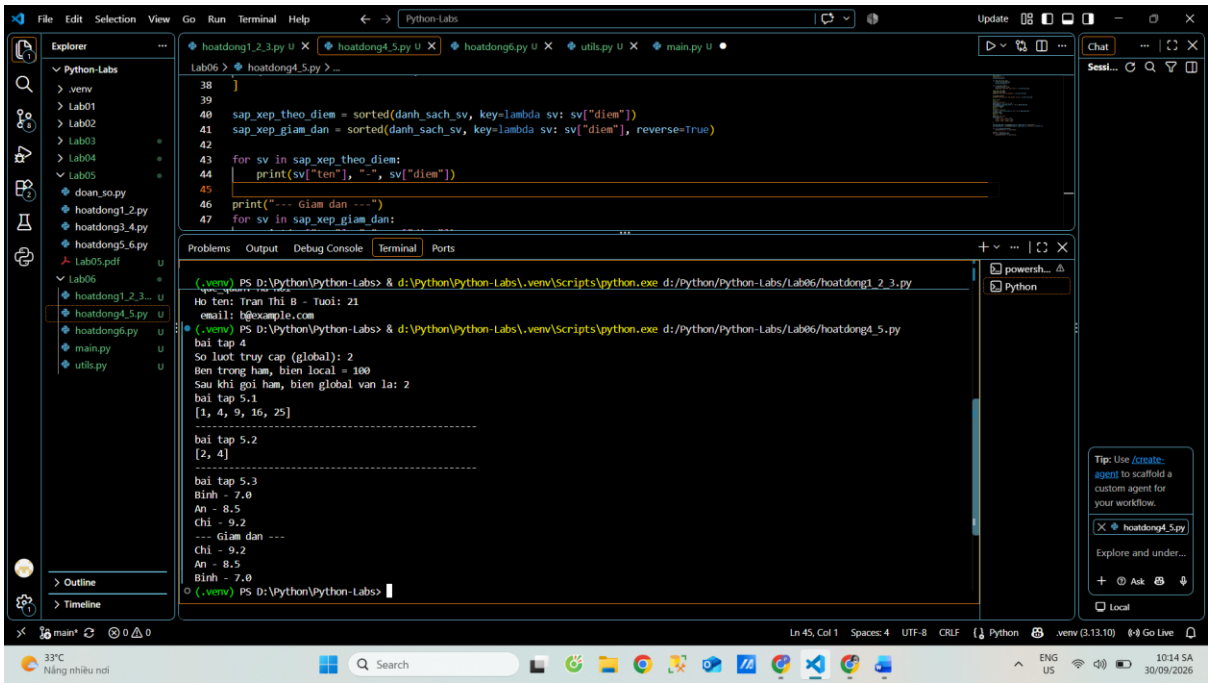
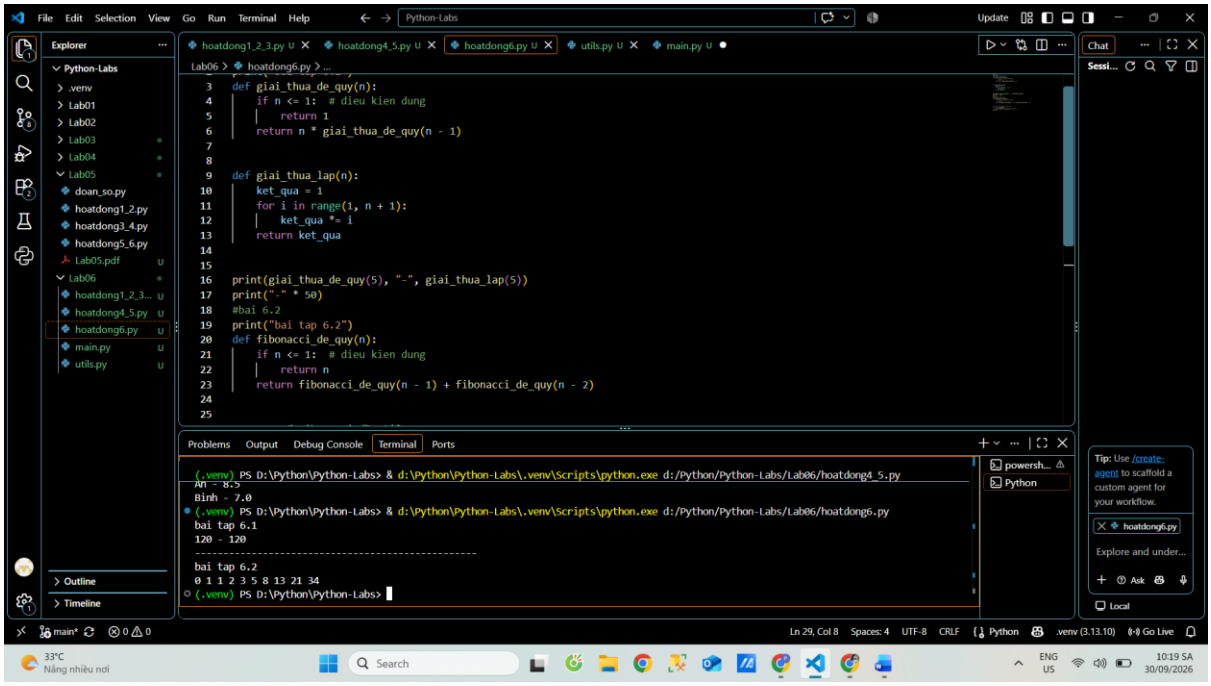
Hoạt động 1_2_3



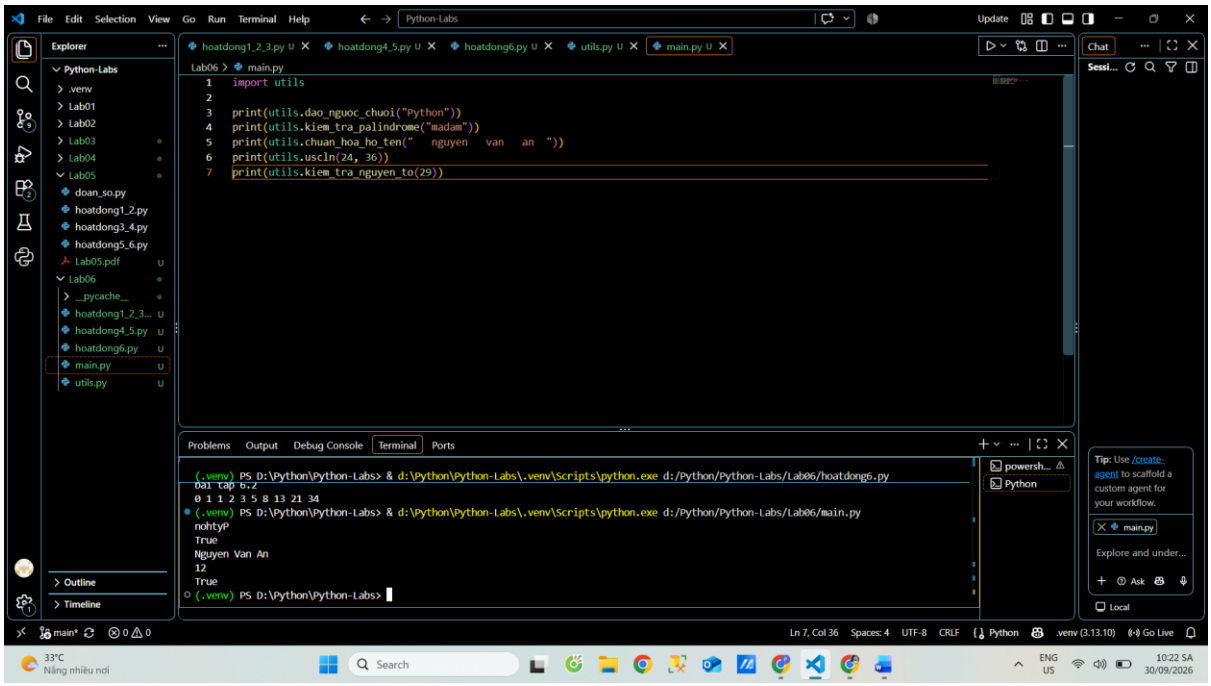
Hoạt động 4_5



Hoạt động 6



Hoạt động 7



Hoạt động 2

1. Khi gọi hàm theo kiểu truyền thống (positional arguments), Python gán dữ liệu dựa vào vị trí thứ tự (tham số thứ 1 nhận giá trị thứ 1, tham số thứ 2 nhận giá trị thứ 2).

2. Khi gọi hàm bằng tham số từ khóa (vd: ten="Dung", lop="CNTT02"), bạn đã chỉ định rõ ràng tên của tham số đi kèm giá trị.

3. Trình biên dịch Python sẽ khớp giá trị dựa trên "tên tham số" thay vì vị trí. Do đó, dù bạn đặt `lop` trước hay `tuoi` trước, Python vẫn biết chính xác giá trị nào thuộc về tham số nào.

Hoạt động 4

1. Khi trong hàm có câu lệnh gán biến `so_luot_truy_cap += 1` (tương đương với “so_luot_truy_cap = so_luot_truy_cap + 1”), Python sẽ mặc định coi `so_luot_truy_cap` là một BIẾN CỤC BỘ (LOCAL) của hàm đó.

2. Để thực hiện phép cộng ở vế phải (`so_luot_truy_cap + 1`), Python cần đọc giá trị hiện tại của biến local này trước. Tuy nhiên, lúc này biến local chưa hề được khởi tạo hay gán giá trị nào bên trong hàm.

3. Vì vậy, Python quăng lỗi `UnboundLocalError` (Cố gắng truy cập một biến cục bộ trước khi nó được gán giá trị.

Hoạt động 5

1. Cách đặt điểm trước tên trong tuple (Buổi 3):

- Bắt buộc phải thay đổi cấu trúc dữ liệu ban đầu hoặc tạo danh sách phụ dạng `(diem, ten)` chỉ để phục vụ việc so sánh.

- Không giữ nguyên được cấu trúc Dictionary hay dữ liệu gốc ban đầu.

2. Vì sao dùng `key=lambda` linh hoạt hơn?

- Giữ nguyên cấu trúc dữ liệu gốc: Danh sách vẫn lưu dạng Dictionary nguyên bản `{"ten": ..., "diem": ...}`.

- Linh hoạt chọn tiêu chí sắp xếp: Bạn có thể dễ dàng đổi sang sắp xếp theo tên (`key=lambda sv: sv["ten"]`), theo điểm (`key=lambda sv: sv["diem"]`), hoặc ghép nhiều tiêu chí phức tạp mà không làm đảo lộn dữ liệu.

Hoạt động 6

So sánh việc sử dụng Đệ quy vs Vòng lặp:

1. Về cú pháp & độ rõ ràng: Đệ quy thể hiện công thức toán học rất ngắn gọn và tự nhiên. Vòng lặp cần tạo thêm biến trung gian (`ket_qua`) để tích lũy.

2. Về hiệu năng & bộ nhớ: Vòng lặp chạy nhanh hơn và tiết kiệm bộ nhớ hơn (O(1)). Đệ quy tốn bộ nhớ cho Stack Frame (Call Stack) để lưu các lần gọi hàm chờ xử lý, nếu `n` quá lớn sẽ gây ra lỗi Stack Overflow.

Vì sao đệ quy Fibonacci lại "tốn kém" và chạy chậm hơn hẳn so với giai thừa đệ quy khi n = 30?

Giải thích:

1. Tính lặp lại không cần thiết: Với đệ quy Fibonacci thuần túy, mỗi lần gọi `fibonacci_de_quy(n)` nó sẽ tách thành 2 nhánh: `(n-1)` và `(n-2)`.Điều này tạo thành một Cây đệ quy (Recursion Tree).

- Ví dụ: Để tính F(5), nó tính F(3) hai lần, F(2) ba lần... Các giá trị # trùng nhau này bị tính toán lặp đi lặp lại rất nhiều lần.

2. Độ phức tạp thời gian tăng theo cấp số nhân:

- Giai thừa đệ quy chỉ gọi hàm n lần -> Độ phức tạp là O(n).

- Fibonacci đệ quy gọi hàm khoảng 2^n lần -> Độ phức tạp là O(2^n).Khi n = 30, số lần gọi hàm lên tới hơn 1 tỷ phép tính, khiến chương trình bị khựng/chậm rõ rệt.

Hoạt động 7

1. Cơ chế tìm kiếm của Python (sys.path): Khi bạn gọi `import utils`, Python sẽ tìm kiếm file `utils.py` theo thứ tự trong danh sách các đường dẫn `sys.path`.

2. Ưu tiên thư mục hiện tại: Vị trí đầu tiên mà Python tìm kiếm chính là Thư mục chứa file script đang được chạy trực tiếp (ở đây là thư mục của `main.py`).

3. Kết quả: Nếu `utils.py` nằm cùng thư mục với `main.py`, Python sẽ lập tức tìm thấy module `utils` và nạp vào chương trình. Ngược lại, nếu nằm ở khác thư mục, Python không tìm thấy file và sẽ quăng lỗi `ModuleNotFoundError: No module named 'utils'`.

https://github.com/2411062203-create/Python-Labs/tree/main/Lab06